

Can small language models running in the browser identify and prevent data leakage in LLM prompts — and does the answer change with larger models, or with a different kind of model?

AI-DLPP research prototype · draft v3 · 17 September 2026 · *author and affiliation to be added* · every figure and table regenerates from the run artifacts with docs/paper/extract-numbers.py and make-figures.py

ABSTRACT. AI-DLPP is a browser extension that checks every prompt bound for ChatGPT, Claude, Gemini or DeepSeek against a natural-language leak policy, entirely on the user's machine, and pseudonymises, redacts or blocks before anything leaves the browser. We ask three questions and report each on accuracy, feasibility, cost and time. (1) **Can small language models, running in real time directly in the browser, identify and prevent data leakage?** Partly. Four 2–4 B-parameter models on WebGPU, with a compiled policy and a regex/validator tier in front of them, fully catch 62–67% of leak-bearing messages while wrongly actioning 25% of clean ones; the model contributes little of that beyond what the regex tier already finds, and at the policy's semantic predicate every in-browser model scores below a trivial capitalisation rule (best F1 0.262 vs 0.776) because it fires on most messages. Only the 2 B model meets the policy's 5-second budget on every message (1.0 s median); larger in-browser models exceed it on 3–70 of 189, and policy-in-context prompting answers 116 of 189 on the 2 B model and none on the others. The cost is hardware, not money: 2.2–3.4 GB of VRAM and a 1.7–3.6 s load. (2) **Does the result differ with larger models?** Yes. The identical method on hosted open-weight models clears the semantic floor on the best of five, DeepSeek, in

every one of three passes (mean F1 0.881) — the two whose sizes are published, 27 and 120 B, do not — catches 84% of leaks at 21% over-blocking, and with reasoning allowed to finish reaches perfect recall on a sixth model; per-call latency is comparable to the best in-browser arm (0.36–1.2 s median), and the whole study cost \$1.22 — \$0.04–0.31 per thousand messages. But hosted models are not shippable under the product's cloud boundary, so the answer points to a local server, and the compiled judge design that transfers there. (3) **Does the answer change again if the model is not generative at all?** Decisively. A System One model (TypeSafe Jev) returns a typed judgment and a probability instead of text: the predicate becomes one probability per message, and each candidate span, proposed by the regex tier, becomes one classification. Over three passes it scores mean F1 **0.963** at the untuned 0.5 default and 0.971 split-half against the 0.776 floor, with precision 1.000 on every pass — not one of 160 clean messages was ever flagged — and perfect ranking, so a 0.12-wide band of thresholds separates the corpus completely. It stops 87% of leak-bearing messages at 25% over-blocking, or 78% at 12%, in 280 ms and at \$0.12 per thousand messages. It is hosted too, so it inherits the same boundary. Seven defects in our own instrumentation and reporting are reported.

1 The three questions

Employees paste confidential material into consumer LLM interfaces. AI-DLPP intercepts each prompt, detects confidential data locally, and applies a policy-driven action — allow, pseudonymise, redact or block — with pseudonymised values restored in the streamed reply. Two design properties are fixed: the taxonomy of "confidential" comes from a natural-language policy document compiled once (a frontier model may be used at compile time, which touches no user data), and **user prompts never leave the browser**. That second property is what makes the questions hard: the only models a shippable build can run are the ones a browser can host.

This paper answers three practical questions, each on the same four dimensions:

- **Q1.** Can small language models, running in real time directly in the browser, identify and prevent data leakage?
- **Q2.** Does the result differ if larger models are used instead of in-browser ones?
- **Q3.** Does it differ again if the model is not generative at all, but returns typed judgments and probabilities?

Accuracy is how well leaks are identified — F1 on entity spans and on the policy's semantic predicate — and, for *prevention*, the share of leak-bearing messages whose every violating span was caught, against the share of clean messages wrongly actioned (spec §6.4's headline pair). **Feasibility** is whether the configuration runs at all, within the policy's per-message budget, and whether it is allowed under the cloud boundary. **Cost** is money per message, or hardware where there is no bill. **Time** is latency per message, measured, and what transfers to other hardware. Table 1 gives the answers in one place; the rest of the paper is the evidence.

Table 1. The three questions on four dimensions. Best shippable configuration against the best hosted generative one and the judgment model, same corpus, same policy, same gold. "Floor" is a rule with no model in it. Leak-prevention / over-blocking are the share of leak-bearing messages fully caught and the share of clean messages that would have been actioned (entity level).

	Q1 · small models in the browser (2–4 B, WebGPU)	Q2 · larger models, hosted (27–120 B where published)	Q3 · typed judgment, hosted (TypeSafe Jev)
Accuracy — semantic predicate	Best F1 0.262 at message level against a floor of 0.776 ; span-wise 0.197 vs 0.571. Every arm is below the floor; precision never exceeds 0.25.	DeepSeek: mean F1 0.881 over three passes (0.811 / 0.905 / 0.927) against 0.776 — clears it every pass; span-wise 0.602 vs 0.571. The other four models never clear it (Qwen3.8 27B sits on the floor: 0.760 / 0.776 / 0.776). With reasoning allowed to finish: recall 1.000, F1 0.929 on answered rows.	Mean F1 0.963 at the untuned 0.5 default over three passes (0.944 / 0.973 / 0.973). Precision 1.000 every pass: 0 false alarms in 160 clean messages, every error a miss. ROC-AUC 1.000; split-half, the honest number, 0.971. No span-wise number: a probability carries no clause.
Accuracy — entity spans	Pipeline with tier 0: F1 0.444–0.486 (compiled 0.444–0.468), of which the regex/validator tier supplies 0.468 by itself; model-only best 0.331, on 116 of 189 answered.	Model-only F1 0.675 (0.664–0.675 across passes); no-label oracle 0.454, label-informed oracle 0.695. With reasoning: 0.697 on all rows, 0.809 on answered rows.	F1 0.551 at 0.5 over the same candidates the regex tier proposes, against 0.459 for that tier alone. Recall is capped by the candidate generator, not by the model.
Prevention	Leaks fully caught 62–67% ; clean messages over-blocked 25% — the same 20 of 81, produced by the regex tier whichever model sits behind it.	DeepSeek: 84% caught / 21% over-blocked. Qwen3.8 27B: 94% / 44%. The gain is recall; over-blocking is set by how eagerly the model fires.	87% caught / 25% over-blocked at 0.5; at 0.85, 78% / 12% , which beats the regex tier on both at once. The threshold is a knob, not a fixed point.
Feasibility	Runs in real Chrome on WebGPU with grammar-constrained JSON. Only the 2 B model answers every message inside the 5 s budget; the 3–4 B models exceed it on 3–70 of 189; policy-in-context prompting never reaches a first token on three of four models.	Not shippable — prompts would leave the machine. Requires a local server running the same open weights. Constrained decoding: 0 malformed responses in 5,580 calls; provider rate limits cost 1.8% of rows; reasoning modes need 8,192-token budgets and minutes per batch.	Not shippable — hosted, the same boundary as Q2. 567 calls, 0 errors, 0 rate limits. One request per message carries the predicate and every candidate question. No TTFT, decode rate or reasoning tokens exist: the transport returns one body.
Cost	\$0 per message. Hardware: 2.2–3.4 GB VRAM, 1.7–3.6 s engine load; the compiled prompt is ~410–450 tokens against 1,570 for policy-in-context.	\$0.04–0.31 per 1,000 messages by model (compiled judge \$0.09, policy-in-context \$0.21, a 2.5× gap); the whole study, 6,502 calls, billed \$1.22. Reasoning: \$0.23 (judge) to \$1.44 (B) per 1,000. Local-server capital cost not measured.	\$0.115 per 1,000 messages , 2,490 input tokens p50; all three passes cost \$0.065. About 1.3× the compiled judge's bill.
Time	2 B model: 1.0 s median , 2.6 s p95 per message. 3–4 B: 2.6–4.5 s median, p95 at the 5 s budget. Policy-in-context on the 2 B model: 3.3 s median over the 116 messages it answers, 4.4 s over all 189.	Compiled judge 0.36–1.2 s median per call ; policy-in-context 0.75–4.5 s. The judge answers in 5–7 tokens, so on a 60 tok/s local server decode is ~0.1 s per message. Reasoning: 3.7–10 s per message at concurrency 3.	280 ms median per message, 328–398 ms p95, one request. About 3.6× faster than the best in-browser arm, and in the band of the fastest hosted judge.

2 What was measured, and how

Compile once in the cloud; check every prompt locally

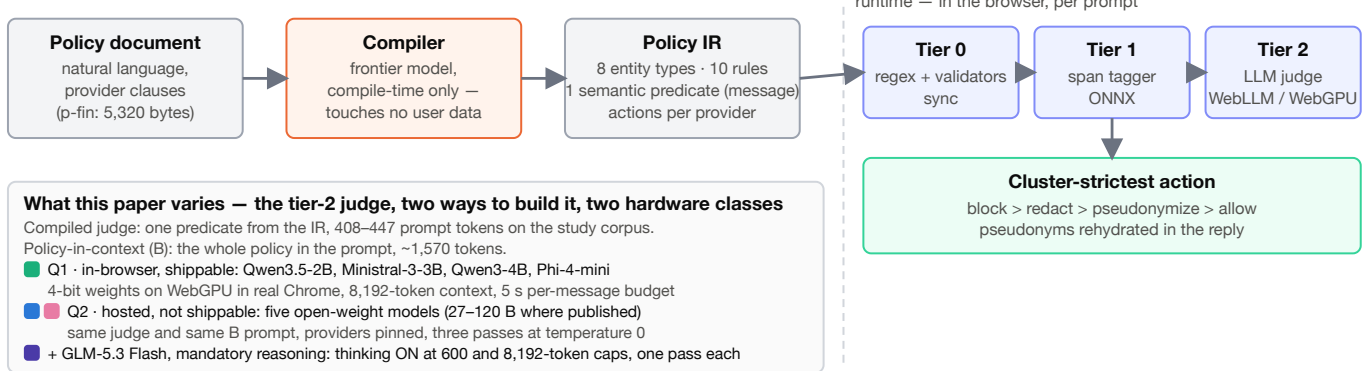


Figure 1. The pipeline, and the two things this paper varies. A policy is compiled once into an intermediate representation; every prompt is then checked locally by three tiers and actioned by the strictest rule in its cluster. The tier-2 judge is built two ways — a compiled, single-predicate judge (408–447 prompt tokens) and a policy-in-context baseline that carries the whole policy (~1,570 tokens) — and is run on two hardware classes: shippable in-browser models, and hosted models that use the identical method and differ only in transport.

2.1 The system

The policy studied, p-fin (5,320 bytes), compiles to eight entity types, ten rules and one message-scoped semantic predicate, `client-relationship-disclosure` — does the message disclose that the firm acts for, is pitching, or is counterparty to a named organisation — whose action is `redact`. Tier 0 is compiled regex with validators and entropy checks; tier 1 a span tagger on ONNX; tier 2 an instruct model answering the predicate through grammar-constrained JSON, quoting a clause and a shorter mention that a two-rung ladder resolves to an action span. Findings are clustered and the strictest action wins. Because the predicate is message-scoped, a message costs exactly one judge call.

2.2 Arms

In-browser (Q1). Four 4-bit models in real Chrome via Playwright — Qwen3.5-2B, Ministral-3-3B, Qwen3-4B, Phi-4-mini — in four configurations each: the full compiled pipeline (tiers 0 and 2), tier 2 alone, policy-in-context (Approach B), and B plus tier 0. Context 8,192 tokens, completion cap 512, and the policy's 5,000 ms per-message budget enforced. Sixteen arms on the study corpus. **Hosted (Q2).** The same judge and the same B prompt sent over OpenRouter to six open-weight models (Table 2), provider pinned, fallbacks disabled, `json_schema` byte with the unchanged schemas, temperature 0, completion cap 600, 60 s timeout. Five models ran with reasoning disabled in three full passes (ten arms, 5,580 calls); GLM-5.3 Flash refuses to disable reasoning (HTTP 400 on all eleven of its providers) and ran thinking-ON instead, at a 600-token cap (judge) and at 8,192 tokens with a 180 s timeout (both families), one pass each. **Judgment (Q3).** TypeSafe System One, model `jev-1.13.0` pinned, one request per message: the policy's predicate as a `noul` returning P(yes), and one choice per candidate span whose options are the IR's own entity definitions plus a not-confidential escape. Candidates are produced in code by tier 0 and the orthographic oracle, so the model never writes a span, it only keeps or rejects one. Three passes.

Table 2. The hosted slate. Provider listing; \$ per million tokens in / out; *US-routed. Only two ids carry a published size (27 B; 120 B with 12 B active).

model	provider	quant.	\$ in	\$ out	hosting
deepseek-v4-flash-0731	DeepInfra	fp8	0.06	0.18	US
qwen3.8-flash	Alibaba	—	0.15	0.47	US*
glm-5.3-flash	BaseTen	fp8	0.15	0.50	US
mistral-small-2603	Mistral	—	0.15	0.60	EU
nemotron-3-super-120b-a12b	DigitalOcean	—	0.30	0.65	US
qwen3.8-27b	Parasail	fp8	0.24	2.20	US

2.3 Corpus, gold, metrics

The corpus has 189 messages with 108 entity gold spans over eight types (API credentials, bank identifiers, client names, DB connection strings, Indian PAN and Aadhaar numbers, internal customer ids, private-key material); 108 messages carry

Table 3. In-browser feasibility on the 189-message corpus — the compiled pipeline and policy-in-context (B) per model, from the run gates. Budget-exhausted = messages on which the judge could not answer inside the policy's 5,000 ms; the p95-TTFT gate requires p95 time-to-first-token under 1,500 ms. Engine load is a warm load in Chrome.

configuration · model	VRAM MB	engine load ms	answered	budget-exhausted	TTFT p95 ms	per-message p50 ms	p95 ms	decode tok/s	p95-TTFT gate
compiled · Qwen3.5-2B	2,245	1,705	189/189	0	1,184	1,018	2,605	46.4	pass
compiled · Ministral-3-3B	2,864	2,620	148/189	45	2,144	2,587	5,025	35.1	killed
compiled · Qwen3-4B	3,432	2,996	119/189	70	2,010	4,523	5,033	27.3	killed
compiled · Phi-4-mini	3,438	3,395	186/189	3	2,031	2,968	4,621	29.6	killed
B · Qwen3.5-2B	2,245	1,680	116/189	73	3,227	4,419	5,021	37.9	killed
B · Ministral-3-3B	2,864	2,707	0/189	189	—	6,153	7,055	—	—
B · Qwen3-4B	3,432	2,956	0/189	189	—	7,588	8,673	—	—
B · Phi-4-mini	3,438	3,054	0/189	189	—	5,878	6,636	—	—

All sixteen arms were driven end to end in real Chrome on WebGPU with grammar-constrained output, on models needing 2.2–3.4 GB of VRAM — six of them (policy-in-context on the three 3–4 B models, with and without tier 0) without the model ever answering a call. Running is not the same as fitting the budget (Table 3, Figure 2). The 2 B model is the only one that answers every message inside 5,000 ms and passes the p95 time-to-first-token gate; Ministral, Qwen3-4B and Phi-4-mini are killed on that gate and exhaust the budget on 45, 70 and 3 messages respectively, which the scorer then counts as misses. Policy-in-context is not feasible in

at least one and 81 are clean. The predicate gold comes from a blind round: two isolated annotators, forbidden the compiled IR, answered *does this message satisfy the predicate* for all 189 rows and agreed on every one; ten rows both marked borderline were excluded as unsettled by the policy text itself, leaving **179 scored rows, 19 positives**. Entity spans are scored one-to-one under an overlap rule. The predicate is scored at message level — the boolean the annotators gave — and, more strictly, span-wise. **Prevention** is entity-level: a leak-bearing message counts as prevented only if its gold span is overlapped by a finding (every leak-bearing message in this corpus carries exactly one); a clean message counts as over-blocked if any finding fires. Every accuracy figure is printed beside a model-free floor: a first-capitalised-words rule scores 0.571 span-wise and 0.776 at message level on the predicate; an orthographic reader scores 0.454 on entity spans, or 0.695 when told each message's span count — a label-informed oracle, named as such. Two limits travel with every number: the corpus generator gives every positive a capitalised organisation name, so the floor cannot miss one; and 19 positives means one item moves recall by 0.053.

3 Q1 — small models, in real time, in the browser

3.1 Accuracy

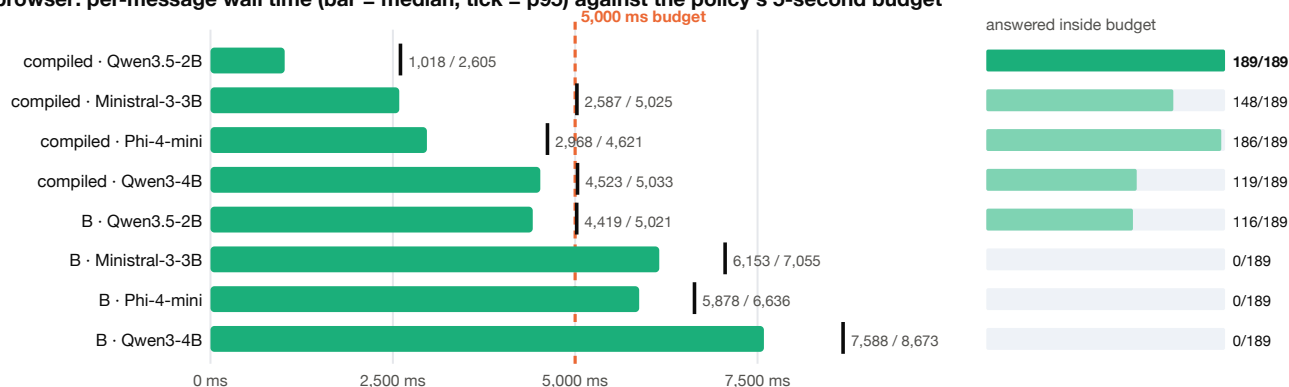
Entity spans. The shippable pipeline scores F1 0.444–0.486 whichever model sits in tier 2, and three configurations whose model answered zero calls score 0.468 from tier 0 alone. The regex and validator tier finds the identifiers; the model adds recall and costs precision. Run without tier 0, the best model-only arm scores 0.331 (B on Qwen3.5-2B, P 0.581 R 0.231, on the 116 of 189 it answers). **The semantic predicate.** No in-browser arm approaches the floor: best F1 0.197 span-wise (compiled Qwen3-4B: 6 tp, 36 fp, 13 fn) and 0.262 at message level, against 0.571 and 0.776. Under exact span matching, eleven of sixteen arms score 0.000 or are undefined (ten under overlap matching). The mechanism is precision (Figure 8): no in-browser arm exceeds 0.25, and Phi-4-mini reaches recall 0.895 by firing on 125 of 179 messages. The single most informative row is a 2 B model placing a finding at exactly [43,59] "Tamarind Grocers", byte-identical to gold, and labelling it an Indian PAN number at confidence 1: these models find the right region and attach the wrong label.

Prevention. The compiled pipeline fully catches 62–67% of the 108 leak-bearing messages (Ministral 72, Qwen3.5-2B 71, Qwen3-4B 70, Phi-4-mini 67) and wrongly actions 25% of the 81 clean ones — the same 20 messages for all four models, because tier 0 produces them. Adding B's findings to tier 0 on Qwen3.5-2B lifts prevention to 73% at 30% over-blocking; B alone manages 23% at 15%. So at the message level a user would experience, the shippable build stops roughly two leaks in three and interrupts one clean prompt in four.

3.2 Feasibility

the browser: its 1,570-token prompt means the 2 B model answers 116 of 189 and the other three answer none at all (median wall 5.9–7.6 s, i.e. the budget). Structured output held — no response was truncated or failed closed, 11 needed a repair pass — but span placement did not always: pooled over the sixteen arms, 41 quoted strings and 53 mentions could not be resolved to a span against 671 that could (12%), and Ministral fails the resolvable-rate gate (0.439 and 0.525 against a floor of 0.8) while emitting 17 findings with labels the policy does not define.

In the browser: per-message wall time (bar = median, tick = p95) against the policy's 5-second budget



Wall time is tier-2 time per message in real Chrome on WebGPU. A message the judge could not answer inside the budget spends the whole budget and is scored as a miss.

Figure 2. Real time, in the browser. Per-message wall time for the compiled pipeline on each model (median and p95) against the policy's 5-second budget, with the share of messages the judge could answer inside it. The 2 B model fits; every larger in-browser model spends part of the corpus at the budget line, and policy-in-context prompting (B) does not fit on three of four models.

3.3 Cost

There is no bill: inference is on the user's GPU. The cost is hardware and load — 2,245–3,438 MB of VRAM per model and a 1.7–3.6 s warm engine load per session — and prompt size, which is what decides whether the model fits the budget at all. The compiled judge's prompt is 408–447 tokens on this corpus against B's 1,570, a 3.5–3.8× gap that the smallest model can absorb and the others cannot.

3.4 Time

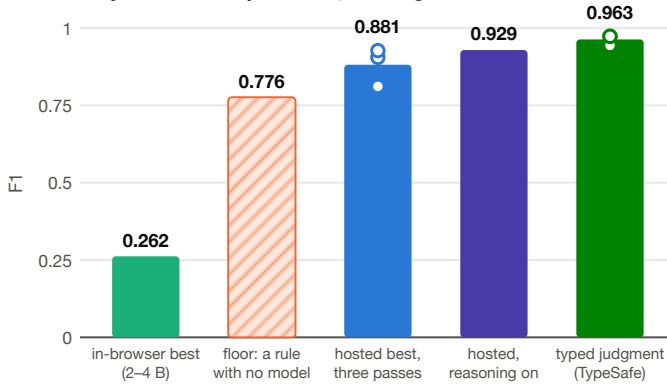
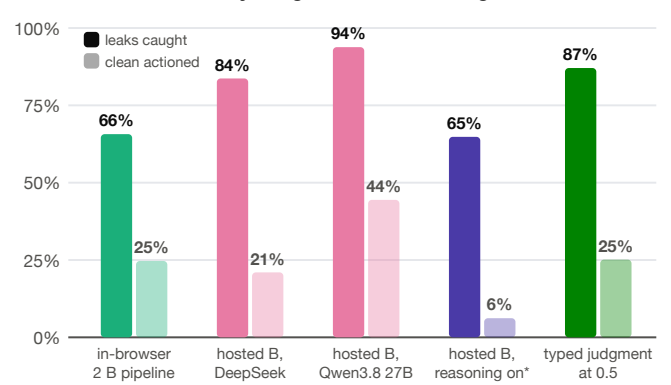
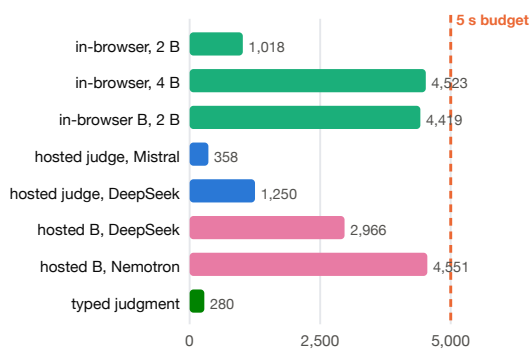
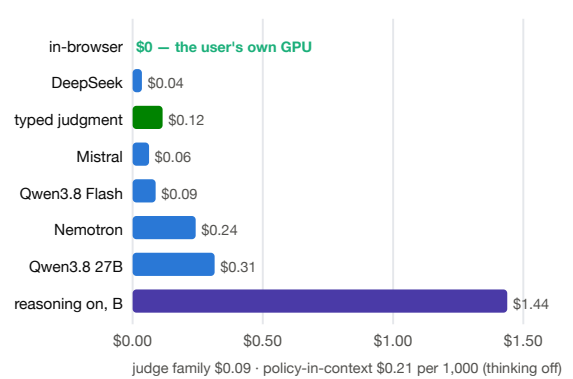
Measured per message, end to end: the 2 B compiled pipeline runs at **1,018 ms median and 2,605 ms p95**, sustaining 46 tokens/s; the 3–4 B models run at 2.6–4.5 s median with p95 pinned at the budget (4.6–5.0 s) and decode at 27–35 tokens/s. B on the 2 B model is 3.3 s median over the 116 messages it answers (4.4 s over all 189). Time-to-first-token p95 is 1,184 ms for the 2 B model and 2,010–2,144 ms for the rest; the compiled prompt is what makes the difference — on a 13-item smoke corpus, over three passes, the compiled prompt reaches p95 TTFT in 556–1,468 ms against 2,734–3,050 ms for B on the 2 B model.

Answer to Q1. Small models in the browser can run a real-time, policy-conditioned check — on the 2 B model, every message inside a second, at no marginal cost. What they *prevent* is mostly what a regex tier prevents: two leaks in three, with one clean prompt in four interrupted. What they cannot do is the semantic judgment the policy asks for — every in-browser model is below a trivial rule at the predicate, because it fires indiscriminately. Identification of structured identifiers: yes. Identification of a client relationship: no. Prevention: partial, and precision-limited.

4 Q2 — larger models instead of in-browser ones

4.1 Accuracy

The identical compiled judge clears the semantic floor on one hosted model of five (Figure 3, Table 4). On DeepSeek it scores 0.811, 0.905 and 0.927 at message level across three passes — mean **0.881 against 0.776**, the last two passes with perfect recall and three or four false positives across 179 messages — and 0.565, 0.615, 0.627 span-wise against 0.571. Where in-browser models fail on precision, hosted ones gain on it (Figure 8). On Qwen3.8 27B the judge sits on the floor in every pass (0.760, 0.776, 0.776 against 0.776); on Qwen3.8 Flash, Nemotron and Mistral it is well below it (0.10–0.48). On entity spans, only B emits findings: its best hosted arm, Nemotron, scores 0.675 model-only — 2.0× the best in-browser model-only arm, above the no-label oracle (0.454) and 0.020 short of the oracle told the span count. With reasoning allowed to finish, GLM-5.3 Flash's judge goes from recall 0.579 to **1.000** (F1 0.929 on the rows it answered, against a 0.743 floor recomputed on the same rows), and its B arm becomes the only one to beat the label-informed oracle (0.809 vs 0.633 on answered rows; 0.697 vs 0.695 on all). **Prevention** (Table 5) improves the way a deployment would feel it: DeepSeek's B arm catches 84% of leak-bearing messages at 21% over-blocking, against the in-browser 62–67% at 25%; Qwen3.8 27B catches 94% but interrupts 44% of clean prompts. Larger models find more; how eagerly they fire is a separate, model-specific property.

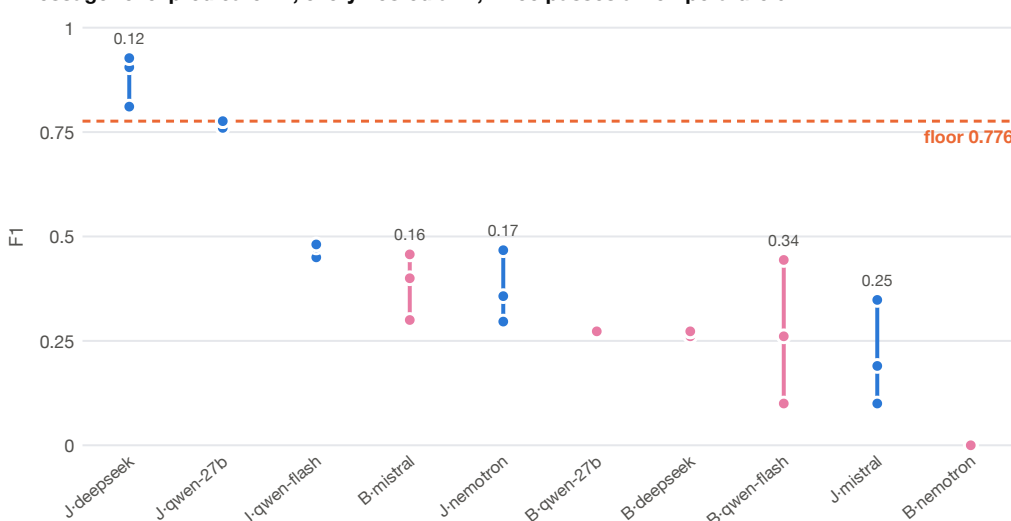
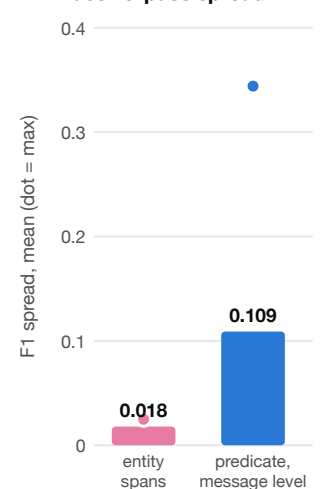
A Accuracy — semantic predicate, message-level F1**B Prevention — leaks fully caught vs clean messages actioned****C Time per message, median (ms)****D Cost per 1,000 messages (USD)**

Same corpus, policy and gold throughout. Hatched = a rule with no model. B = policy-in-context. Typed judgment = TypeSafe at its default 0.5. *Reasoning-on counts 59 unanswered rows as misses.

Figure 3. The four dimensions side by side. (A) Semantic predicate F1 at message level: the best in-browser arm, the model-free floor, the best hosted arm (three passes) and the hosted model with reasoning allowed to finish. (B) Prevention at entity level — leaks fully caught against clean messages over-blocked — for the shippable in-browser pipeline and the hosted policy-in-context arms. (C) Per-message time for the best in-browser arm and hosted arms against the 5 s budget. (D) Cost per thousand messages, hosted, thinking off; in-browser is \$0.

Table 4. Every hosted arm, every pass. F1 at the predicate scored span-wise (floor 0.571) and at message level (floor 0.776), and at entity spans (oracle 0.695 / 0.454), and at message level (floor 0.776). Bold blue exceeds the floor or oracle. J = compiled judge, B = policy-in-context. Unanswered = rows the provider never served.

arm	predicate, span-wise (floor 0.571)			predicate, message level (floor 0.776)			entity spans (oracle 0.695 / 0.454)			unanswered
	p1	p2	p3	p1	p2	p3	p1	p2	p3	
J-deepseek	0.565	0.615	0.627	0.811	0.905	0.927	—	—	—	4/1/1
J-qwen-27b	0.561	0.571	0.571	0.760	0.776	0.776	—	—	—	0/0/0
J-qwen-flash	0.340	0.348	0.340	0.450	0.474	0.481	—	—	—	0/0/0
J-nemotron	0.400	0.357	0.222	0.467	0.357	0.296	—	—	—	0/25/20
J-mistral	0.000	0.320	0.091	0.100	0.348	0.190	—	—	—	0/0/0
B-nemotron	—	—	—	0.000	0.000	0.000	0.675	0.669	0.664	0/2/0
B-deepseek	0.000	0.000	0.000	0.261	0.273	0.273	0.570	0.595	0.583	19/4/4
B-qwen-27b	0.000	0.000	0.000	0.273	0.273	0.273	0.585	0.577	0.589	0/0/0
B-mistral	0.227	0.205	0.095	0.400	0.457	0.300	0.526	0.527	0.545	0/0/0
B-qwen-flash	0.174	0.000	0.148	0.261	0.100	0.444	0.520	0.542	0.531	13/0/0

A Message-level predicate F1, every hosted arm, three passes at temperature 0**B Pass-to-pass spread**

Providers pinned, temperature 0 — what moves between passes is routing, not the sampler. Number above a stem = spread > 0.05. J = judge, B = policy-in-context.

Figure 4. One pass is not a measurement. Message-level predicate F1 per hosted arm across three passes at temperature 0, providers pinned; spread is 6x larger for the predicate (0.109) than for entity spans (0.018). The instability is where the inaccuracy is.

Two properties of the hosted result matter for anyone repeating it. Every call was at temperature 0 with the provider pinned, yet message-level F1 spread by 0.109 on average and 0.344 at most between passes (Figure 4), while entity-span F1 spread by 0.018: a single pass is not a measurement of classification quality, and the instability sits exactly where the inaccuracy does. And 68 of 3,780 rows in the first two

Table 5. Prevention, entity level. Share of the 108 leak-bearing messages whose gold span was caught (each carries exactly one), and share of the 81 clean messages with any finding — the shippable in-browser pipeline per model, and the hosted policy-in-context arms (mean over three passes). The compiled judge emits no entity findings by construction.

arm	leaks fully caught	prevention	clean messages actioned	over-blocking
in-browser compiled · Qwen3.5-2B	71/108	66%	20/81	25%
in-browser compiled · Ministral-3-3B	72/108	67%	20/81	25%
in-browser compiled · Qwen3-4B	70/108	65%	20/81	25%
in-browser compiled · Phi-4-mini	67/108	62%	20/81	25%
in-browser B + tier 0 · Qwen3.5-2B	79/108	73%	24/81	30%
hosted B · qwen-27b (3-pass mean)	—	94%	—	44%
hosted B · qwen-flash (3-pass mean)	—	93%	—	57%
hosted B · deepseek (3-pass mean)	—	84%	—	21%
hosted B · mistral (3-pass mean)	—	77%	—	31%
hosted B · nemotron (3-pass mean)	—	73%	—	28%
hosted B · glm-5.3-flash, reasoning on (all rows; 59 unanswered)	70/108	65%	5/81	6%

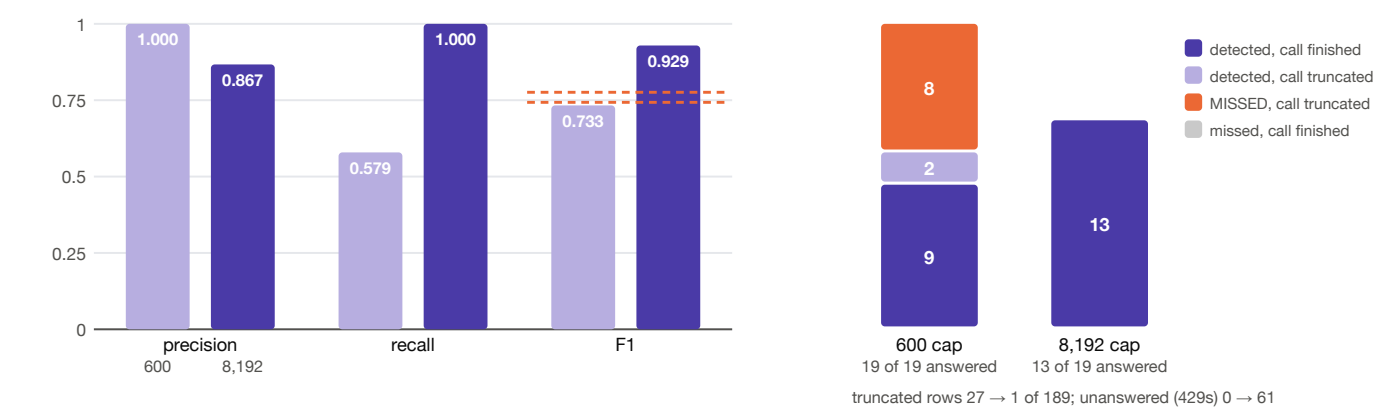
4.2 Feasibility

Hosted inference is **not a shippable configuration**: the product's cloud boundary forbids prompts leaving the machine, and these calls did. The question they answer is what a local server running the same open weights would deliver — a DGX-Spark-class box is the intended target — and on that reading the operational findings are encouraging. Provider-side constrained decoding with the unchanged schemas produced **zero malformed or schema-invalid responses in 5,580 calls** (5,577 parsed, 3 truncated at the cap); reasoning: {enabled: false} was hon-

passes were never answered — rate-limited after up to 16.6 s of backoff — which the scorer charges as misses; re-scoring each arm and floor over the rows it answered changes one thinking-off verdict (DeepSeek pass 1 moves from 0.565 vs 0.571 to 0.578 vs 0.565).

oured on every one of those calls by every provider. Two things were not encouraging. Shared-pool rate limiting is concentrated (DeepSeek's two arms absorbed 638 of 808 retries) and cost 1.8% of rows outright, and it struck the reasoning run harder: 31% of GLM's rows at 8,192 tokens were lost to 429s, and its B-family calls ran 46–54 s where they completed, which put a full thinking-ON slate near ten hours of wall-clock and is why the other five models were not run thinking-ON. Reasoning also needs the budget: at 600 tokens 22.2% of GLM's answers truncated and every one of its eight missed positives was a truncated call; at 8,192 the missed list is empty but the cap was still hit (once on the judge; 22 calls on 20 rows for B).

A GLM judge, reasoning on: 600 vs 8,192-token cap (message level, answered rows) B The gold positives, by call outcome



Dashed: floors recomputed over the same answered rows (0.776 at 600, 0.743 at 8,192). Every miss at 600 was a truncated call, with no clean misses; at 8,192 the missed list is empty.

Figure 5. Reasoning needs room. GLM-5.3 Flash's compiled judge with reasoning on, at a 600-token and an 8,192-token completion cap: (A) precision, recall and F1 at message level over the rows each run answered, with the floor recomputed on the same rows; (B) the gold positives by what happened to the call — at 600 every miss is a truncation; at 8,192 there are none.

4.3 Cost

Table 6. Spend, reconciled. Seven ledger segments joined and checked against the key endpoint; final key reading \$1.29213 of \$10, residual +\$0.073, of which \$0.028 is a probe ledger the authors overwrote by reusing a run id.

ledger segment	calls	billed
probe	11	\$0.00121
ceiling-01 window 1	1,457	\$0.13645
glmon-01	225	\$0.05572
ceiling-01 window 2	768	\$0.19269
ceiling-02+03	3,756	\$0.51381
thinkon-01 (aborted)	9	\$0.00350
thinkonglm-01	276	\$0.31599
total (seven reconciled segments)	6,502	\$1.21937

The whole study — 6,502 calls across seven reconciled ledger segments — billed **\$1.219** (Table 6, above). Per thousand messages, thinking off, the five models range from **\$0.036** (DeepSeek) to **\$0.315** (Qwen3.8 27B), an 8.8× spread for identical work; the compiled judge averages \$0.086 and policy-in-context \$0.212, a 2.5× gap that is prompt size and does not shrink with hardware. Reasoning is dearer: \$0.233 per thousand for the judge and \$1.439 for B, and the GLM pair alone cost 37% of the entire thinking-off slate's \$0.844. These are provider prices; a local server replaces them with a capital cost this study did not measure. The price-table estimate feeding the spend guard overshot billed cost by 68% on the thinking-off segments and 48% over the whole study — safe for a guard, useless for budgeting.

4.4 Time

Per call, pooled over three passes (Table 7, Figure 6): the compiled judge runs at **357–1,209 ms median** (Mistral to Nemotron) and policy-in-context at 752–4,487 ms on the same models, with time-to-first-token p95 of 563–4,736 ms for the judge (up to 5,796 for B). That is the same order as the best in-browser arm's 1,018 ms median — hosted models are not faster per call; they are more accurate at the same speed. The judge is uniformly faster than B on every model because it answers in **5–7 completion tokens** against 58–113; on a 60 tok/s local server that is ~0.1 s of decode per message against 1–1.9 s. Time-to-first-token and decode rate are provider facts and do not transfer; token counts do. Reasoning is slow: 11.5 minutes for the judge and 31.8 minutes for B over 189 messages at concurrency 3, i.e. 3.7–10 s per message.

Hosted, per-call wall time p50 (ms), pooled over three passes — compiled judge vs policy-in-context on the same model

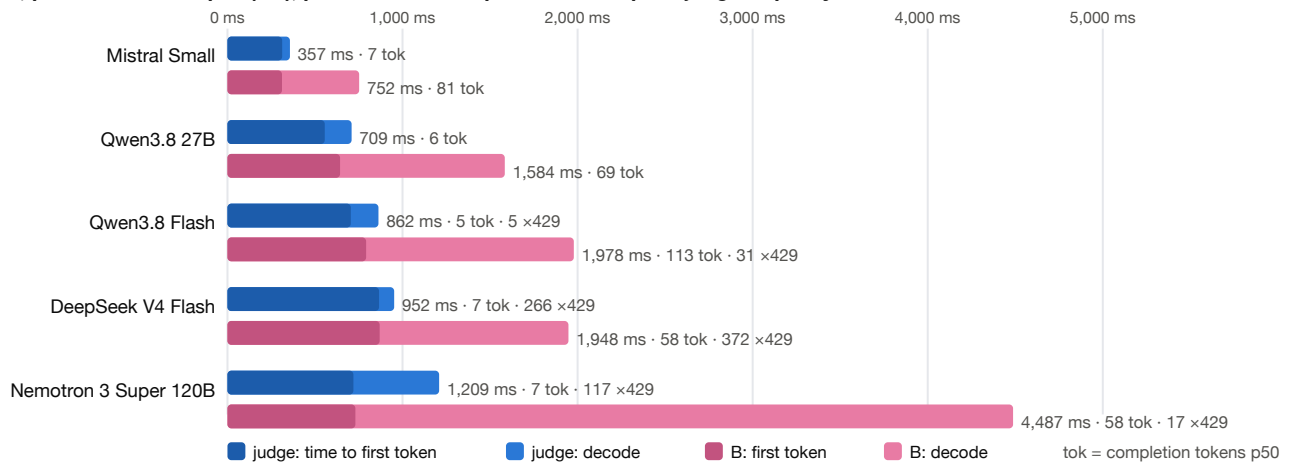


Figure 6. Per-call wall time, hosted, judge vs policy-in-context on the same model. Pooled over three passes. The dark segment is time to first token; the remainder is decode. The judge's answer is 5–7 tokens, so its call is almost entirely first-token latency; B's 58–113 tokens make decode the larger share. Labels give completion tokens and 429 retries.

Table 7. Latency per hosted arm, pooled over three passes (ms). *Item wall is over answered rows only. The "s @60 tok/s" column is the transferable number: completion tokens at a local-server decode budget. The decode-rate column is deliberately absent (§6).

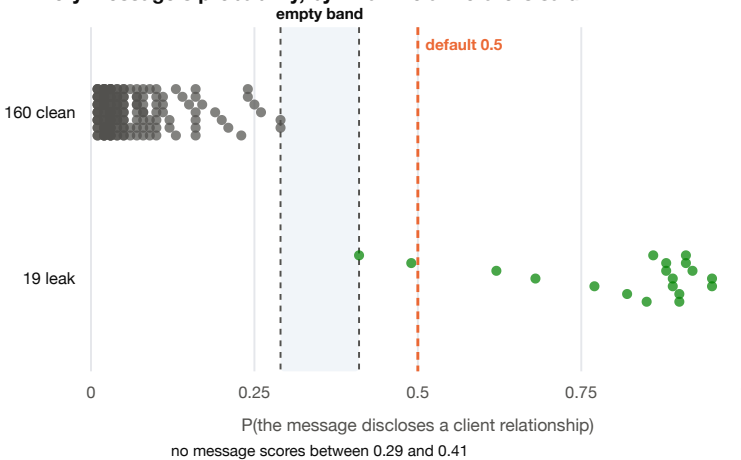
arm	calls	TTFT p50	TTFT p95	call wall p50	item wall p50*	compl. tok p50	s @60 tok/s	429s
J-mistral	567	312	563	357	358	7	0.12	0
J-qwen-27b	567	556	871	709	710	6	0.10	0
B-mistral	568	311	592	752	752	81	1.35	0
J-qwen-flash	567	704	4,736	862	954	5	0.08	5
J-deepseek	561	866	3,058	952	1,250	7	0.12	266
J-nemotron	522	719	2,314	1,209	1,221	7	0.12	117
B-qwen-27b	567	643	984	1,584	1,584	69	1.15	0
B-deepseek	541	869	2,975	1,948	2,966	58	0.97	372
B-qwen-flash	555	792	5,796	1,978	2,123	113	1.88	31
B-nemotron	565	731	2,392	4,487	4,551	58	0.97	17

Answer to Q2. Yes, on accuracy: the same method on the best larger open-weight model goes from below a trivial rule to clearly and repeatably above it at the semantic predicate (one model of five; the two with published sizes do not), doubles model-only entity F1, lifts prevention from two leaks in three to five in six at lower over-blocking (21% against 25%), and with reasoning reaches perfect recall. Not on time: per-call latency is comparable to the best in-browser arm. On cost: cents per thousand messages hosted, or a local server. On feasibility: hosted is not allowed, so Q2's answer is really "a local server running these weights would" — and the compiled judge, at 5–7 output tokens, is the design that carries there.

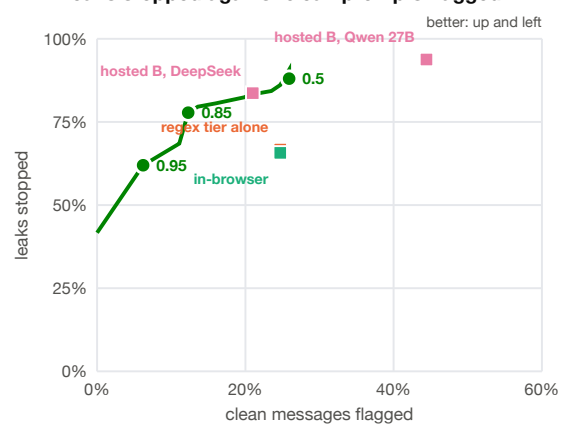
5 Q3 — a typed judgment instead of generated text

Both arms so far ask a model to write JSON and then recover a span from what it wrote. A System One model cannot do that: it returns a typed answer and a probability. The decomposition inverts — code proposes candidate spans, the model judges them, code places them — and two of this study's standing difficulties go with it. There is no clause to fail to quote, so the span ladder is unnecessary and no finding is ever unresolved. And the answer is a probability, so one run yields a curve where every other arm yielded a single point.

A Every message's probability, by what the annotators said



B Leaks stopped against clean prompts flagged



Left: pass 1, all 179 scored messages, dots stacked where they collide. Right: the judgment threshold swept 0 to 1 (line) against every other arm's single operating point (squares).

Figure 7. What a judgment arm looks like. (A) Every one of the 179 scored messages on the probability axis in pass 1, split by what the annotators said; the shaded band is empty, so any threshold inside it separates the corpus perfectly. (B) Sweeping that threshold traces a curve through the prevention / over-blocking plane, where every other arm in this study is a single fixed point.

5.1 Accuracy

At the untuned 0.5 default the judgment arm scores F1 0.944, 0.973 and 0.973 at the semantic predicate — mean **0.963** against the 0.776 floor and DeepSeek's 0.881 (Table 8, Figure 3A). Precision is 1.000 on every pass: **not one of the 160 clean messages was flagged, in any pass.** Every error is a miss — two, one and one of the 19 positives — and every missed positive still outranks every negative. ROC-AUC is 1.000 on all three passes: the highest-scoring clean message sits at 0.29 and the lowest-scoring leak at 0.41, so any threshold in that 0.12-wide band scores the

corpus perfectly (Figure 7A). Because such a threshold is chosen on the rows it is then scored on, the number we report is the split-half estimate, which picks the threshold on one half of the messages and spends it on the other: **0.971**, identical across the three passes.

Calibration is not what a probability usually promises. 160 of 179 messages fall below 0.3 and every one of them is a true negative; every message above 0.4 is a true positive; the middle is empty. This is a separating score rather than a graded belief — better for routing, and not to be read as a frequency.

Stability is the second result, and for this study the more surprising one. Message-level F1 moved by 0.029 across the three passes, against 0.109 on average and 0.344 at most for the hosted generative arms at temperature 0 (§4.1,

Table 8. The judgment arm, three passes. The predicate at the 0.5 default, with false alarms counted against the 160 clean messages and misses against the 19 positives; the split-half F1 chooses its threshold on one half of the messages and scores the other. Entity rows are at the same 0.5 and at 0.85, with the regex tier alone beside them.

the judgment arm (jev-1.13.0)	pass 1	pass 2	pass 3
predicate F1 at 0.5	0.944	0.973	0.973
precision / recall	1.000 / 0.895	1.000 / 0.947	1.000 / 0.947
false alarms (of 160 clean)	0	0	0
misses (of 19 leaks)	2	1	1
F1, split-half threshold	0.971	0.971	0.971
ROC-AUC	1.000	1.000	1.000
entity prevention / over-blocking at 0.5	88% / 26%	86% / 25%	87% / 25%
the same at 0.85	78% / 12%	77% / 14%	79% / 15%
regex tier alone, no model	67% / 25%	67% / 25%	67% / 25%
candidates kept wrongly / rejected wrongly	21 / 7	21 / 7	19 / 7
cost per 1,000 messages	\$0.115	\$0.115	\$0.115
median ms per message	280	280	280

5.2 Prevention, and what the judgment adds

At the entity level the arm stops 87% of leak-bearing messages while actioning 25% of clean ones, against 67% and 25% for the regex tier alone working from the same candidates. Raised to 0.85 it stops 78% at 12% over-blocking: the first configuration in this study that beats the regex tier on both counts at once (Figure 7B). Since the model only keeps or rejects what code proposed, what it adds is countable directly: of the 101 tier-0 candidates that touch no gold span it rejected 80, and of the 105 that do touch one it wrongly rejected 7. That is the whole trade, and it is why precision rises without recall collapsing. The ceiling is the candidate list: a span no regex proposed cannot be found here at all, which is why the tier-0 row sits beside every entity number.

5.3 Feasibility, cost and time

567 calls across the three passes: 189 of 189 answered every time, zero errors, zero rate limits, zero retries. One request per message carries the predicate and every candidate question — three questions at the median — because the documented billing charges the document once however many questions ride on it. Median 280 ms per message (p95 328–398 ms), about 3.6× faster than the best in-browser arm and inside the band of the fastest hosted judge. Cost is \$0.115 per thousand messages at 2,490 input tokens p50; all three passes together cost \$0.065, about 1.3× the compiled judge's rate. Four columns the other arms fill stay empty here and are marked as such rather than zeroed: time to first token and decode rate (the transport returns one body, not a stream), reasoning tokens, and span-wise predicate F1, since a probability carries no clause.

Answer to Q3. Yes, and it is the largest single change in the study. Dropping generation entirely — a typed judgment over spans that code proposes — clears the semantic floor by a wider margin than any generative arm, at perfect precision, with a separating score that makes the operating point a deliberate choice rather than a decoder's accident, and with pass-to-pass movement an order of magnitude below the generative arms. It is faster and it is cheap. It is also hosted, so it answers the same question Q2 did: what a local server could deliver. The design that transfers is the shape, not the vendor — code proposes, a small judgment model disposes.

6 Where the models fail, and what that decides

At every scale the binding constraint is classification, not span extraction: a 2 B model places spans byte-exactly, the hosted arms resolve 99.2% of findings cleanly, and what moves between passes and separates arms is the label. Two consequences for the design follow. The two ways of building the judge are complementary rather than ranked — the compiled judge wins the semantic predicate (0.927 vs 0.457 at best) and is 2× cheaper and 2–4× faster, while policy-in-context is the entity finder (it is the only family that emits spans) — so a deployment runs both, which the tiered design already does. And the in-browser tier keeps its job: tier 0 supplies the identifier catching that the shippable build actually delivers, at 1 s per message on a 2 B model, while the semantic predicate is what a larger model on a local server would add. The judgment arm fails differently, and usefully: it cannot invent a span, so its errors are confined to keeping a candidate that should have been rejected or rejecting one that should have been kept, and both are counted directly in §5.2 rather than inferred from a score.

Figure 4). The predicate probability was identical on 88 of 189 messages and never moved by more than 0.070; 407 of 417 candidate labels were identical across all three passes.

Message-level predicate: precision vs recall, every arm

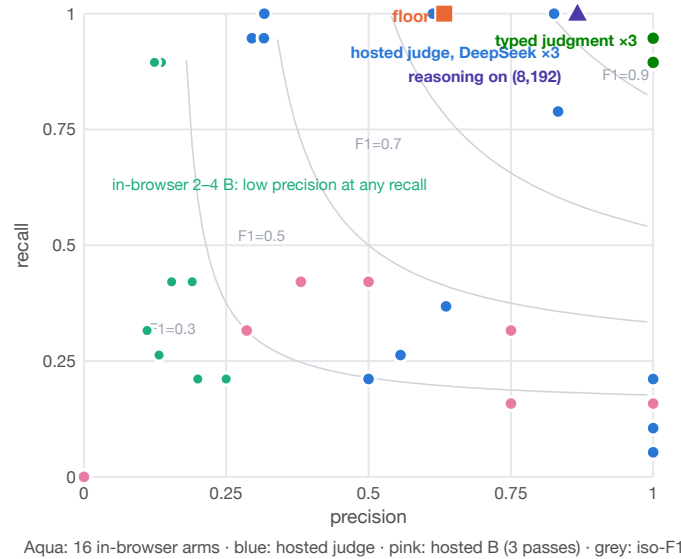


Figure 8. Why the in-browser arms fail: precision. Precision against recall at the message-level predicate for every arm. In-browser arms reach recall only by firing on most messages; the hosted judge separates on precision at the same recall; the floor has perfect recall by construction of this corpus.

7 Limits, and defects in our own instrumentation

One policy, one predicate, one corpus with a generator leak that gives the capitalisation floor perfect recall; 19 positives; in-browser and hosted arms in different harnesses; hosted quantisation is the provider's; the reasoning result is one model, one pass, with 31% of rows lost; and the study measured OpenRouter endpoints, not the local server it argues toward. Seven defects in the instrumentation and reporting were found by re-deriving published numbers from artifacts and by adversarial mutation review, and are stated rather than fixed silently: the hosted completion cap (600) did not match the browser cap (512) — 12 of 5,580 completions reached 512 and 3 truncated at 600; the reasoning arm could not be measured at that cap; two wall clocks (per-call excludes backoff, per-item includes it and rows with no call, inflating p95 up to 2.5×); unserved rows charged as misses; a message-level gold scored span-wise, which had reversed an earlier headline; and a decode rate over n instead of $n-1$ tokens, 14–22% biased on 5–7-token answers over 9.6–491 ms windows; and, in this paper's own first draft, a cost basis that counted GLM's 600-token reasoning run as thinking-off, which printed the judge-to-B cost ratio as 2.05× and the GLM share as 35% (2.5× and 37% here). The driver, the scorer and ledger together, and the per-item call library were mutation-tested in three campaigns (23 of 23, 20 of 20 and 27 of 32 mutants killed, the five survivors being equivalent or fixture-bound) and the apps/eval suite grew from 791 to 906 tests in those campaigns and stands at 908 at the time of writing.

8 Conclusion

Q1. Small language models can run a real-time, policy-conditioned leak check directly in the browser — on a 2 B model, every message inside a second, at no marginal cost and 2.2 GB of VRAM. What that check prevents is what its regex tier prevents: about two leak-bearing messages in three, at the price of one clean prompt in four. The semantic identification the policy asks for is beyond every in-browser model tested, not for want of finding the text but for want of precision about what it is. **Q2.** The result does differ with larger models, on accuracy specifically: the identical method on the best hosted open-weight model, DeepSeek, clears the semantic floor in every pass — one model of five; the two with published sizes, 27 and 120 B, do not — catches five leaks in six at lower over-blocking, and reaches perfect recall when reasoning is given room — at per-call latency comparable to the

browser's best and at cents per thousand messages. Those models cannot ship under the cloud boundary, so the practical answer is a local server running them, with the compiled judge — five to seven output tokens per message — as the design that transfers. **Q3.** The largest change comes from dropping generation altogether. A typed judgment over candidate spans that code proposes scores 0.963 at the un-tuned default and 0.971 on held-out thresholds, flags none of the 160 clean messages in any of three passes, separates the corpus completely, and moves by 0.029 between passes where the generative arms moved by up to 0.344 — at 280 ms and \$0.12 per thousand messages. It is hosted as well, so it inherits Q2's boundary and Q2's answer: run it where you control it. Read the three together and the shape of a shippable build is visible: the browser's regex tier catches identifiers in real time for nothing, and a judgment model on a machine the organisation owns decides the questions the policy actually asks. Every number here regenerates from artifacts; the study's most useful by-product may be the seven ways its own instrumentation was found wrong.

Reproducibility. Repository `orkancelikhisar/ai-dlpp`, branch `feat/tier2-judge`: `driver pnpm -C apps/eval ceiling, scorer ceiling:score, ledger ceiling:ledger`; the judgment arm is `typesafe` and `typesafe:score`, and `SIH_TS_DRY_RUN=1` runs its whole path with no key; this paper's data and figures regenerate with `docs/paper/extract-numbers.py` and `make-figures.py` from `runs/` and `corpora/generated/`. Full records in `docs/research/`. The API key was revoked after the study; its final reading is recorded, not reproducible.